

Try-On Workflow Documentation

Nandini Bhimineni

Overview

The Try-On Workflow in Raritone is designed to support AI-powered virtual clothing trials using realtime backend communication, optimized image handling, and scalable API architecture.

The workflow allows users to:

- Upload images
- Select products
- Initiate AI try-on processing
- Receive realtime updates
- View optimized try-on results

The backend architecture is designed for future AI scalability and realtime interaction.

Try-On API Flow

Main APIs

Start Try-On Request

Endpoint:

POST /api/tryon/start

Purpose:

- Initiate AI try-on processing
- Send user and product data
- Begin realtime workflow

Check Try-On Status

Endpoint:

GET /api/tryon/status/:id

Purpose:

- Check processing status
- Retrieve current workflow state

Possible statuses:

- Pending
- Processing
- Completed
- Failed

Get Try-On Result**Endpoint:**

GET /api/tryon/result/:id

Purpose:

- Retrieve processed try-on image
- Return optimized Cloudinary URL

Delete Try-On Session**Endpoint:**

DELETE /api/tryon/:id

Purpose:

- Remove old try-on data
- Clean up unused media

API Request Flow

User Uploads Image



Frontend Sends Try-On Request



Backend Receives Request



Validation Middleware Executes



AI Processing Starts



Realtime Updates Sent



Processed Image Uploaded To Cloudinary



Optimized Result Returned To Frontend

API Response Flow

Frontend sends:

- User image
- Selected clothing
- Product information

Backend responds with:

- Request accepted
- Processing status
- Final try-on result

Request Example

POST /api/tryon/start

```
{  
  "userImage": "user-image-url",  
  "productId": "12345"  
}
```

Initial Response Example

```
{  
  "success": true,  
  "status": "processing",  
  "tryOnId": "TRYON123"  
}
```

Final Result Response Example

```
{  
  "success": true,  
  "status": "completed",  
  "resultImage": "cloudinary-url"
```

}

Realtime Socket.IO Flow

Socket.IO is used to provide realtime updates during AI processing.

Workflow:

Frontend Sends Try-On Event

↓

Backend Receives Event

↓

AI Processing Starts

↓

Realtime Status Updates Sent

↓

Frontend Receives Final Result

Socket Events

Event	Purpose
connection	User connection established
tryon-request	Send try-on request
tryon-status	Realtime processing updates
disconnect	User disconnected

Benefits Of Realtime Workflow

- No repeated API polling
- Faster user experience
- Instant processing updates
- Scalable realtime communication

AI Processing Flow

The backend architecture is prepared for future AI integration.

Planned AI workflow:

User Image + Product Image

↓

Backend Validation

↓

AI Processing Service



Virtual Try-On Generation



Processed Image Output



Cloudinary Upload



Optimized Result Delivery

Cloudinary Result Flow

After AI processing:

- Generated image uploaded to Cloudinary
- CDN optimized URL generated
- Frontend receives optimized result

Cloudinary optimizations include:

- Automatic compression
- Automatic modern formats
- CDN acceleration
- Optimized image delivery

Example optimization parameters:

- w_500
- h_500
- c_fill
- f_auto
- q_auto

Validation & Security

The try-on workflow includes:

- Joi validation
- JWT protection
- Helmet security

- Rate limiting
- Centralized error handling

This ensures:

- Secure APIs
- Protected user data
- Reliable request handling

Postman Verification

The APIs were verified using Postman.

Verified:

- Request handling
- Response flow
- Validation responses
- Protected routes
- Realtime backend integration

Future Improvements

Possible future enhancements:

- AI queue systems
- GPU processing servers
- Avatar generation
- Recommendation engine
- Distributed realtime scaling
- Redis Pub/Sub integration

Conclusion

The Raritone Try-On Workflow provides a scalable and realtime-ready backend architecture for future AI-powered virtual try-on experiences.

The workflow combines:

- Realtime communication
- Optimized media handling

- Scalable APIs
- Secure backend architecture
- CDN optimized image delivery

to deliver an efficient and future-ready try-on system.

Schema Documentation & Database Design

Overview

The Raritone backend uses MongoDB with Mongoose schemas to manage user data, AI try-on workflows, avatar information, and product compatibility.

The database architecture is designed to support:

- AI virtual try-on systems
- Realtime processing
- Scalable backend workflows
- Optimized data relationships

1. Measurement Schema

Purpose

The Measurement schema stores body measurements required for:

- Size recommendations
- Personalized fitting
- AI try-on compatibility
- Avatar generation support

Main Fields

```
{  
  userId,  
  height,  
  weight,  
  chest,  
  waist,  
  hip,
```

```
    shoulder,  
    inseam,  
    shoeSize,  
    createdAt  
}
```

Relationships

- Linked to User schema using `userId`
- Used by TryOn system
- Supports recommendation engine

AI Usage

Measurement data helps:

- Improve fitting accuracy
- Support AI body analysis
- Personalize clothing recommendations

2. TryOn Schema

Purpose

The TryOn schema stores:

- Virtual try-on requests
- Processing status
- Generated result images
- Try-on history

Main Fields

```
{  
  userId,  
  productId,  
  userImage,  
  processedImage,  
  status,  
}
```



```
    createdAt  
  }
```

Relationships

- Linked to User schema
- Linked to Product schema
- Connected to Cloudinary image storage

AI Usage

Used for:

- AI processing tracking
- Virtual try-on history
- Realtime processing updates

3. Avatar Schema

Purpose

The Avatar schema stores AI-generated avatar information for virtual fitting experiences.

Main Fields

```
{  
  userId,  
  avatarImage,  
  bodyType,  
  skinTone,  
  gender,  
  avatarModel,  
  createdAt  
}
```

Relationships

- Linked to User schema
- Connected to TryOn workflow
- Supports recommendation engine

AI Usage

Avatar data is used for:

- AI avatar generation
- Personalized try-on rendering
- Virtual body modeling

4. ProductMapping Schema

Purpose

The ProductMapping schema helps determine clothing compatibility for users and avatars.

Main Fields

```
{  
  productId,  
  category,  
  clothType,  
  supportedBodyTypes,  
  compatibility,  
  avatarCompatibility,  
  createdAt  
}
```

Relationships

- Linked to Product schema
- Connected to Avatar schema
- Used by recommendation systems

AI Usage

Used for:

- Product-avatar matching
- AI recommendation systems
- Compatibility analysis
- Virtual fitting optimization

Database Relationships

User

↓

Measurement

↓

Avatar

↓

TryOn

↓

ProductMapping

↓

Product

Database Design Advantages

Scalability

Supports future AI expansion and large-scale user handling.

Modularity

Each schema handles a specific backend responsibility.

AI Readiness

Prepared for:

- AI try-on systems
- Recommendation engines
- Avatar processing

Realtime Support

Integrated with realtime Socket.IO workflows.

Schema Verification

All schemas were verified through:

- Backend startup testing
- MongoDB model validation
- API integration testing

Verified:

- Schema exports

- Backend compatibility
- Database structure consistency

Future Improvements

Possible future enhancements:

- Advanced indexing
- Redis caching
- AI analytics storage
- Distributed database scaling
- Sharding support

Conclusion

The Raritone database architecture provides a scalable and AI-ready backend foundation for virtual try-on systems, realtime workflows, avatar generation, and recommendation engine support.

The schema design ensures:

- Organized data management
- Scalable relationships
- Future AI compatibility
- Efficient backend operations